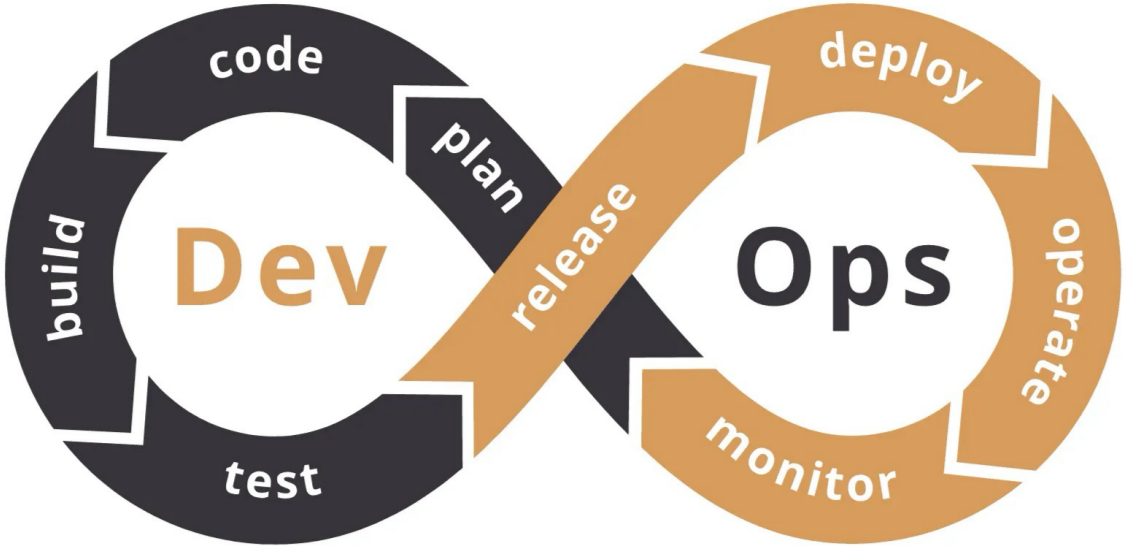
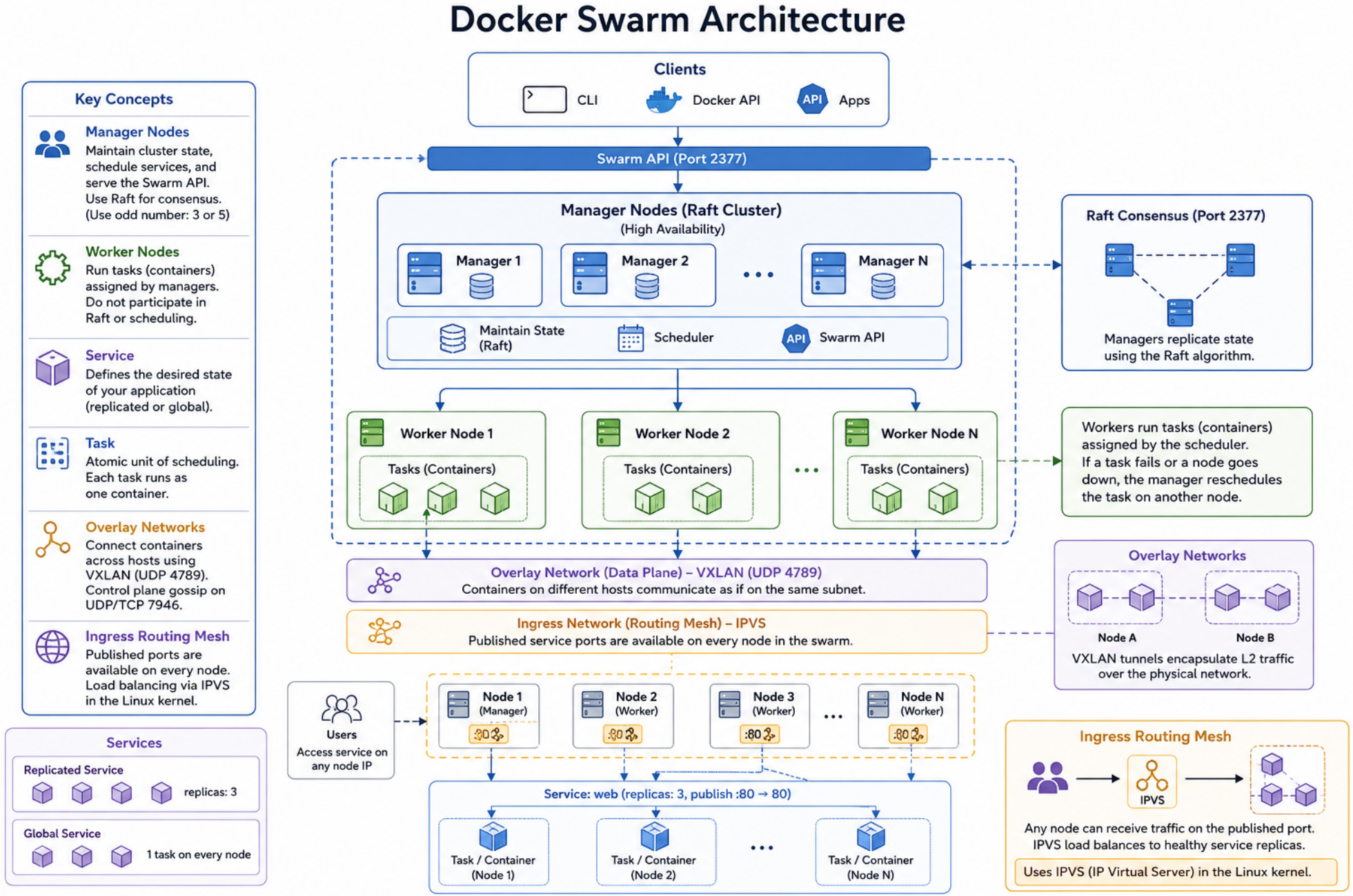


Trainer: Nilesh Ghule

Container Orchestration with Docker Swarm



Nilesh Ghule

Kubernetes Architecture

Kubernetes Objects

Pod
Smallest deployable unit that can contain one or more containers.

ReplicaSet / Deployment
Ensures the desired number of Pod replicas are running.

Service
Stable networking endpoint to access a set of Pods.

ConfigMap / Secret
Store configuration data and sensitive information.

PersistentVolume (PV)
Piece of storage in the cluster provisioned by admin or dynamically.

PersistentVolumeClaim (PVC)
Request storage by a user for a Pod.

Namespace
Virtual cluster within a physical cluster.

Key Principles

- **Declarative:** Define desired state, K8s reconciles actual state.
- **Self-healing:** Restarts, reschedules, recreates failed containers.
- **Scalable:** Easy horizontal scaling and rollouts.
- **Extensible:** Rich ecosystem via APIs and controllers.

Control Plane (Master Node(s))

etcd
- Consistent and highly available key-value store
- Stores the cluster state

kube-api-server
- Frontend for Kubernetes API
- Validates requests
- Updates and retrieves cluster state

kube-scheduler
- Watches for newly created Pods without a node
- Selects the best node for the Pod to run on

cloud-controller-manager (optional)
- Interacts with cloud provider APIs
- Manages load balancers, volumes, routes, etc.

kube-controller-manager
- Runs controller processes
- Node, Replication, Endpoints, ServiceAccount, etc.

Users / Clients

- kubectl
- UI (Dashboard)
- API Clients

Worker Nodes

Worker Node 1

kubelet
- Ensures containers are running in a Pod
- Reports node & Pod status to API server

kube-proxy
- Maintains network rules on the node
- Enables Service networking

Pod

Worker Node 2

kubelet
- Ensures containers are running in a Pod
- Reports node & Pod status to API server

kube-proxy
- Maintains network rules on the node
- Enables Service networking

Pod

Worker Node N

Add-ons (Typically running as Pods)

CoreDNS · Metrics Server · Ingress Controller · Dashboard · Logging / Monitoring · ...

Networking

- All nodes run a CNI (Container Network Interface) plugin.
- Pods get their own IP addresses and can communicate across nodes.
- Services provide stable virtual IPs and DNS names.

Core DNS

- Cluster DNS service discovers Services and other resources.
- Provides DNS resolution for Services and Pods.

Ingress (Optional)

- Provides HTTP/HTTPS routing to Services based on host/path.
- Typically implemented via Ingress Controller.

How it works (High Level Flow)

1. User submits request (kubectl/ API)
2. API Server validates and stores desired state in etcd
3. Scheduler assigns Pod to a suitable node
4. Kubelet runs Pod containers via container runtime
5. Kube-proxy & CNI enable networking and connectivity
6. Service is accessible to users via stable IP / DNS name

Container Runtime

containerd · CRI-O · Other OCI Runtimes

Storage

Persistent Volumes (PV) → Persistent Volume Claims (PVC)

API / HTTPS Communication · Control Plane to Nodes (Watch / Heartbeat) · Node Local Communication · Networking / Data Plane · Container

Nilesh Ghule

Kubernetes Pod and Container – How They Work Together

Analogy: A Container is a single tenant, while a Pod is the shared apartment they live in.

1. The Abstraction Layer

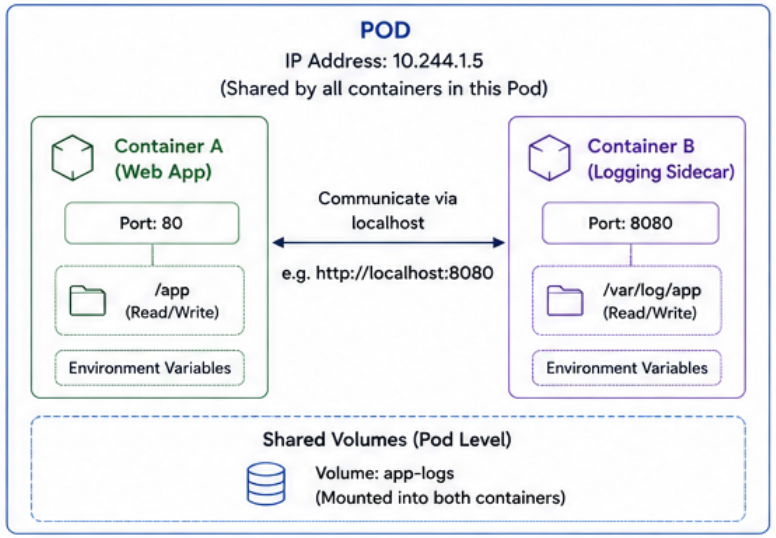
Container
Knows nothing about Kubernetes.
It only knows about its own application code, environment variables, and local files.

Pod
The smallest atomic unit of deployment in Kubernetes.
Kubernetes schedules Pods, not individual containers.
If a Pod moves, all containers inside it move together.

2. Isolation vs. Sharing

Resource	Isolated or Shared?	What it means
Filesystem (Root FS)	Isolated	Each container has its own read-only root filesystem. They cannot see each other's files unless a volume is mounted.
Process Space	Isolated	Container A cannot see or stop the processes running in Container B.
Network	Shared	Same IP, MAC, network namespace. Communicate via localhost.
Storage (Volumes)	Shared (via Volumes)	Pod owns the storage. Mount the same volume into containers to share data.

Example Pod: A Web Application with Sidecar



3. Network Aspect (Shared)

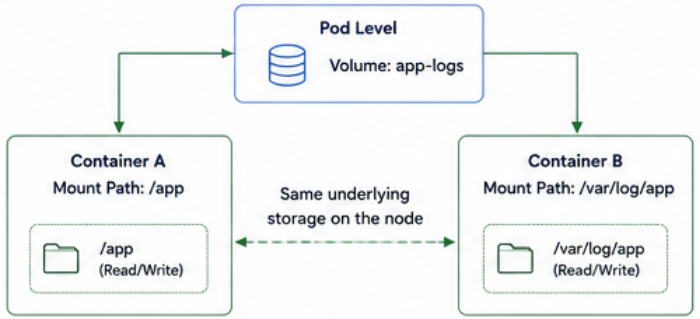
Same Network Namespace
All containers share the same IP, MAC address, and network namespace.

Localhost Communication
Containers can talk to each other using localhost:port.
Example: Frontend → http://localhost:8080

Port Conflicts
Since they share the same IP, two containers cannot listen on the same port.
If both try to use port 80, one will fail.

External World
To the rest of the cluster, the Pod appears as a single entity with a single IP address.

4. Storage Aspect (Shared via Volumes)



5. Lifecycle and Scaling (Coupled)

Co-scheduling
All containers in a Pod are always scheduled on the same Node.
They cannot be split across nodes.

Fate Sharing
They live and die together.
If the Pod is deleted, fails, or is rescheduled, all containers in it are stopped and destroyed together.

Scaling
You scale Pods, not containers.
To run more instances, Kubernetes creates more Pods (each with its set of containers).

Summary Checklist

Resource	IP Address	Ports	Storage	File System	Processes
Shared or Isolated?	Shared	Shared	Shared (via Volumes)	Isolated	Isolated
What it means in practice	All containers share one IP; talk via localhost.	Containers cannot use the same port number.	Can read/write to the same folder if configured.	Individual system files are locked to each container.	Containers cannot see each other's active processes.

Key Takeaway

A Pod is a logical host for one or more containers that need to work closely together.

They share the same network and storage, but remain isolated in filesystem and processes.

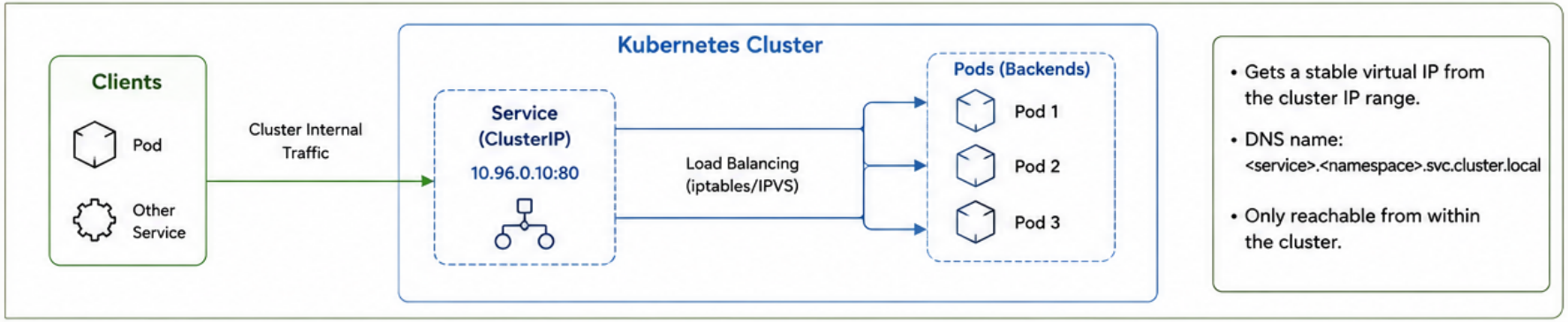
Nilesh Ghule

Kubernetes Service Types – How They Work

1. ClusterIP

Default

Exposes the Service on a cluster-internal IP. Accessible only from within the cluster.

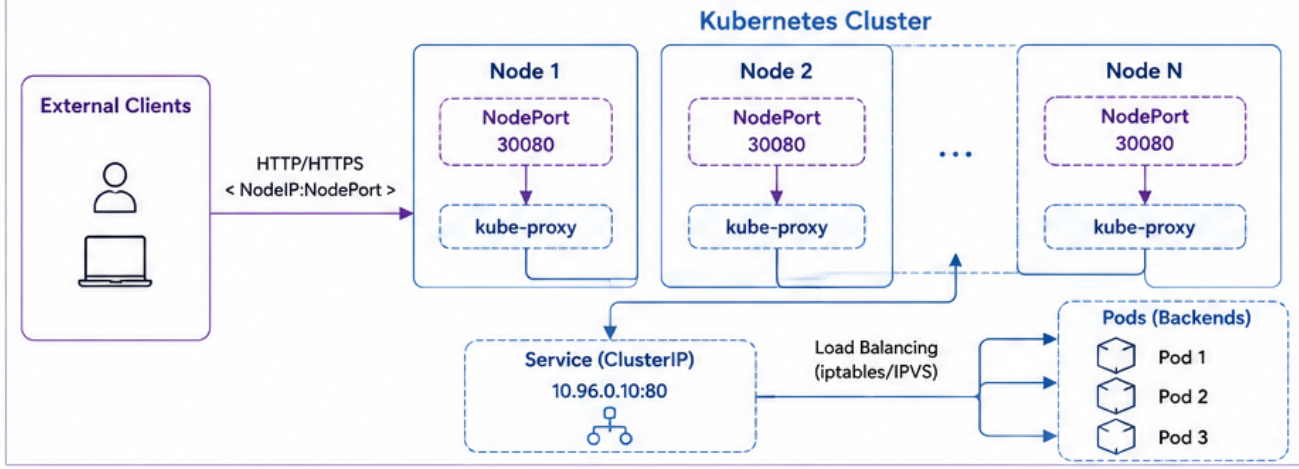


- Gets a stable virtual IP from the cluster IP range.
- DNS name: <service>.<namespace>.svc.cluster.local
- Only reachable from within the cluster.

2. NodePort

Externally on each node's IP

Exposes the Service on a static port on each node's IP. External clients can access the Service via <NodeIP>:<NodePort>.

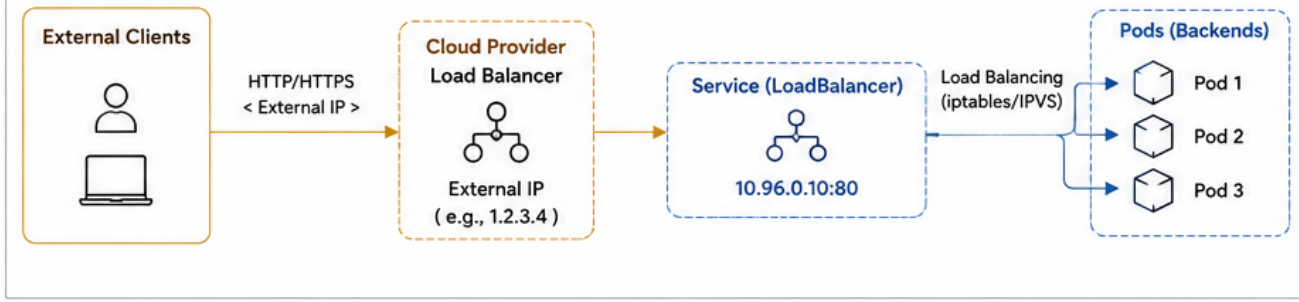


- A static port (30000–32767) is opened on every node.
- kube-proxy forwards traffic from <NodeIP>:<NodePort> to the Service (ClusterIP).
- Reachable from outside the cluster using any node's IP and the NodePort.

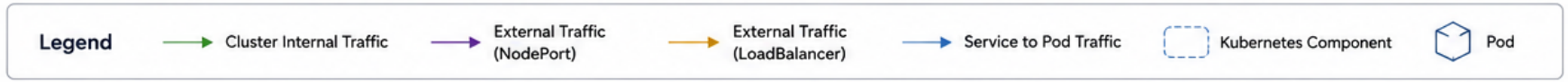
3. LoadBalancer

Externall via Cloud Load Balancer

Exposes the Service externally using a cloud provider's Load Balancer. The LB gets an external IP that routes to the Service.



- Cloud provider provisions a Load Balancer (e.g., AWS ELB, GCP GLB, Azure LB).
- Gets an External IP (or hostname) that clients use to access the Service.
- Traffic flows: Client → LB → Service (ClusterIP) → Pods.



Nilesh Ghule

Thank You!

Nilesh Ghule

<nilesh@sunbeaminfo.com>